Fiche de restitution — Vanlife « Bretagne Sauvage » · version 2

Objet : demande de réservation **réelle** du circuit Bretagne Sauvage — site public, serveur et base de données, e-mails automatiques, espace client et back-office. Extension de la version 1.0 (prototype à envoi simulé, fiche `FICHE_RESTITUTION_V1.md`), conservée dans l'historique git. **Lancement :** Docker Desktop démarré, `npm install` (racine et `backend/`), puis `npm run demarrer` → site http://localhost:5173 · back-office `#/admin` · e-mails reçus sur http://localhost:8025 (détails dans le README).

Ce que fait l'application

- **Site** : marque VANLIFE.BZH, séjour pilote, **carte du circuit** (SVG, ≈ 645 km), **programme détaillé** (route, nuit, activités), **fiche du van**, prix et prestations, **questions fréquentes**, galerie. Contenu lu en base.
- **Demande** : choix parmi les **départs réels** (vans restants, départs complets grisés), récapitulatif, envoi, référence `VL-AAAA-NNNN`. Mêmes règles de saisie côté site et côté serveur (code partagé).
- **E-mails** : accusé de réception au client, notification à l'équipe, changement de statut, réponses. **File d'envoi** en base : si la messagerie tombe, la demande est acceptée et l'e-mail repart seul (1 min, 5 min, 15 min, 1 h).
- **Espace client** par **lien personnel** reçu par e-mail : étapes de la demande, échange avec Vanlife, annulation tant que le séjour n'est pas confirmé. Lien perdu : référence + e-mail → nouveau lien **envoyé par e-mail**.
- **Back-office** (`#/admin`) : liste filtrable, détail, changement de statut (avec e-mail au client), note interne, réponse au client, gestion des départs, export tableur. Rôles administrateur / gestionnaire.

Choix techniques et sécurité

- React 19 + TypeScript · Fastify 5 · PostgreSQL 18 + Prisma 6 · Nodemailer + Mailpit · Docker. Types et règles partagés entre site et serveur (`src/domain/`).
- **Pas de double réservation** : départ verrouillé pendant le comptage des places (5 demandes simultanées → 1 acceptée).
- Mots de passe **Argon2id** ; session en cookie `httpOnly` + `SameSite=Strict` ; contrôle d'origine (CSRF) ; connexion limitée à 5 essais/min ; aucun compte par défaut. Secrets (sessions, liens) stockés en empreinte SHA-256.
- Le lien de suivi et l'e-mail du client ne passent jamais dans l'adresse des requêtes (absents des journaux) ; e-mails échappés (pas d'injection de code) ; export CSV protégé contre l'injection de formules.

Outils utilisés

React · TypeScript · Vite · TanStack Query · Fastify · Prisma · PostgreSQL · Docker · Nodemailer · Mailpit · Vitest + Testing Library + MSW · Puppeteer · ESLint · Wikimedia Commons · Claude (assistance à la conception et au code ; chaque résultat vérifié par les tests, le parcours dans Chrome et la relecture des captures).

Problème observé pendant les essais et correction

- **Constat :** un test du parcours échouait **environ une fois sur dix**, seulement dans la suite complète, jamais seul.
- **Cause :** le premier test de chaque fichier charge une page « à la demande » ; sur une machine occupée ce chargement dépassait le délai d'attente des tests (1 s).
- **Correction :** pages pré-chargées avant les tests et délai porté à 5 s (`src/test/configuration.ts`).
- **Vérification :** 4 lancements complets consécutifs sans échec.
- *Autres corrections issues des essais :* un visiteur non connecté recevait « requête invalide » au lieu de « non connecté » (il découvrait le format de l'API) ; la déconnexion ne ramenait pas à la page de connexion ; suivi par référence + e-mail jugé trop faible pour permettre une annulation → remplacé par le lien personnel ; libellés coupés sur smartphone ; étiquettes de la carte superposées ; texte de FAQ citant un champ inexistant.
- **Essai complet par l'étudiant (13/11/2026)** : demande → e-mails → espace client → back-office → confirmation : fonctionnel.
- **Essai par une autre personne :** *à compléter* — testeur : ..... · difficulté observée : ..... · correction : .....

Vérifications réalisées

`npm run verifier` (site) : lint 0 erreur · **143/143 tests** · build OK. `cd backend && npm run verifier` : **107/107 tests** sur une base de test séparée. `npm run demo` : parcours complet dans Chrome à 390 et 1366 px. Essais réels : panne de messagerie et reprise, conversation client ↔ équipe, annulation, lien trafiqué refusé.

Limites restantes

E-mails capturés par Mailpit (aucun envoi réel sans fournisseur SMTP). Application non publiée en ligne. Paiement volontairement hors périmètre. Photos libres de droits en attendant les visuels Vanlife. Distances estimées ; carte schématique. Pas encore de purge automatique des données anciennes. Pas de test avec un lecteur d'écran réel.

Éléments que Vanlife doit confirmer

1. **Prix** (1 390 € pour 2) et répartition inclus / à prévoir.
2. **Lieux de nuit** et partenaires (dont « secteur Iroise ») ; **activité nautique** et solution si elle est annulée.
3. **Départs** : saison, jours, nombre de vans (hypothèse : chaque samedi d'avril à octobre 2027, 2 vans).
4. **Conditions** : caution, animaux, âge et ancienneté de permis, annulation après confirmation.
5. **Van** : modèle et équipement exact ; adresse d'expédition des e-mails et nom de domaine.